

VibeVolts Project Documentation

Jules the Pirate Engineer

September 19, 2025

Contents

1	Summary	2
2	Python Code Listings	2
2.1	all_demos.py	2
2.2	common.py	3
2.3	constants.py	4
2.4	demo1.py	5
2.5	demo2.py	5
2.6	demo3.py	7
2.7	demo4.py	8
2.8	demo_exclusion_debug_print.py	9
2.9	demo_exclusion_table.py	9
2.10	demo_fixedpoints.py	10
2.11	demo_lambertian.py	11
2.12	demo_pointing_plot.py	12
2.13	demo_pointing_sequence.py	12
2.14	demo_pointing_vectors.py	14
2.15	demo_sky_scan.py	14
2.16	generate_log_spherical_points.py	15
2.17	generate_report.py	17
2.18	lambertian.py	18
2.19	plotting_3d.py	19
2.20	plotting_vectors.py	20
2.21	pointing.py	21
2.22	pointing_vectors.py	23
2.23	propagation.py	24
2.24	radiometry_calcs.py	26
2.25	radiometry_data.py	28
2.26	simulation.py	30
2.27	visibility.py	32

1 Summary

VibeVolts is a Python-based simulation toolkit for space environment modeling. It provides a set of tools to initialize, propagate, and analyze the state of various space-based and ground-based assets. The project's primary goal is to create a fast and efficient simulation tool for modeling satellite and observatory visibility of objects in space, with a focus on performance by leveraging vectorized operations with NumPy.

The toolkit is structured into several core modules, each responsible for a specific part of the simulation, such as orbit propagation ('propagation.py'), visibility calculations ('visibility.py'), and satellite pointing ('pointing.py'). The entire simulation state is managed within a single dictionary, 'simulation_data', which is initialized by the 'simulation.py' module.

VibeVolts includes a number of demonstration scripts that showcase its features. These demos cover everything from basic satellite propagation to more complex scenarios like sky scans and exclusion zone calculations. The 'all_demos.py' script provides a convenient way to run all the demonstrations.

2 Python Code Listings

2.1 all_demos.py

```
1 import plotly.graph_objects as go
2
3 # Import all the demo functions
4 from demo1 import demo1
5 from demo2 import demo2
6 from demo3 import demo3
7 from demo4 import demo4
8 from demo_fixedpoints import demo_fixedpoints
9 from demo_exclusion_table import demo_exclusion_table
10 from demo_pointing_plot import demo_pointing_plot
11 from demo_exclusion_debug_print import demo_exclusion_debug_print
12 from demo_lambertian import demo_lambertian
13 from demo_sky_scan import demo_sky_scan
14 from demo_pointing_vectors import demo_pointing_vectors
15 from demo_pointing_sequence import demo_pointing_sequence
16
17 def run_all_demos():
18     """
19     Runs all demo functions and displays their plots.
20     This function is designed to be called from a Jupyter notebook
21     or another script.
22     """
23     demo_functions = [
24         demo1,
25         demo2,
26         demo3,
27         demo4,
28         demo_fixedpoints,
29         demo_exclusion_table,
30         demo_pointing_plot,
31         demo_lambertian,
32         demo_sky_scan,
33         demo_pointing_vectors,
34         demo_pointing_sequence,
35     ]
36
37     print("--- Running All Demos ---")
38     for func in demo_functions:
39         print(f"\n... Executing {func.__name__} ...")
40         result = func()
41         if isinstance(result, go.Figure):
42             result.show()
43
44     # Also run the non-plotting demo
45     demo_exclusion_debug_print()
46
47     print("\n--- All demos complete. ---")
48
49 if __name__ == '__main__':
```

```
50 run_all_demos()
```

2.2 common.py

```
1 import numpy as np
2 from datetime import datetime
3 from typing import Dict, Any
4 from astropy.coordinates import solar_system_ephemeris
5
6 # Adjusting import paths for the new project structure
7 from propagation import readtle, propagate_satellites
8 from simulation import initializeStructures
9
10 def initialize_standard_simulation(start_time: datetime) -> Dict[str, Any]:
11     """
12     Initializes a standard simulation with a predefined set of satellites.
13
14     This function consolidates TLE data, initializes the main data structure,
15     and propagates satellites to their initial positions at the specified start
16     time. This ensures the returned simulation state is fully populated with
17     positions and radially-outward pointing vectors.
18
19     Args:
20         start_time: The timezone-aware datetime object for the simulation start.
21
22     Returns:
23         The fully initialized and propagated simulation data dictionary.
24     """
25     # Consolidated TLE data from all demos
26     tle_data = """ISS (ZARYA)
27 1 25544U 98067A   25209.52203988   .00012111  00000+0  22159-3 0  9991
28 2 25544   51.6412 254.9961 0006733   98.4322 261.6813 15.49493393462383
29 NOAA 19
30 1 33591U 09005A   25209.38959223   .000000100  00000+0  97987-4 0  9993
31 2 33591   99.1533 244.3362 0013327 101.3725 258.7562 14.12510122810029
32 HST
33 1 20580U 90037B   25208.83160218   .000000113  00000+0  35999-4 0  9990
34 2 20580   28.4695 177.8391 0001259 138.5273 221.5822 15.09326468 23453
35 GEO-01
36 1 90001U 25001A   25209.50000000   .000000000  00000-0  00000-0 0  9991
37 2 90001   0.0500   0.0000 0001000   0.0000   0.0000 1.00270000   12
38 GEO-02
39 1 90002U 25001B   25209.50000000   .000000000  00000-0  00000-0 0  9992
40 2 90002   0.0500   36.0000 0001000   0.0000   0.0000 1.00270000   13
41 GEO-03
42 1 90003U 25001C   25209.50000000   .000000000  00000-0  00000-0 0  9993
43 2 90003   0.0500   72.0000 0001000   0.0000  45.0000 1.00270000   14
44 GEO-04
45 1 90004U 25001D   25209.50000000   .000000000  00000-0  00000-0 0  9994
46 2 90004   0.0500 108.0000 0001000   0.0000  90.0000 1.00270000   15
47 GEO-05
48 1 90005U 25001E   25209.50000000   .000000000  00000-0  00000-0 0  9995
49 2 90005   0.0500 144.0000 0001000   0.0000 135.0000 1.00270000   16
50 GEO-06
51 1 90006U 25001F   25209.50000000   .000000000  00000-0  00000-0 0  9996
52 2 90006   0.0500 180.0000 0001000   0.0000 180.0000 1.00270000   17
53 GEO-07
54 1 90007U 25001G   25209.50000000   .000000000  00000-0  00000-0 0  9997
55 2 90007   0.0500 216.0000 0001000   0.0000 225.0000 1.00270000   18
56 GEO-08
57 1 90008U 25001H   25209.50000000   .000000000  00000-0  00000-0 0  9998
58 2 90008   0.0500 252.0000 0001000   0.0000 270.0000 1.00270000   19
59 GEO-09
60 1 90009U 25001I   25209.50000000   .000000000  00000-0  00000-0 0  9999
61 2 90009   0.0500 288.0000 0001000   0.0000 315.0000 1.00270000   10
62 GEO-10
63 1 90010U 25001J   25209.50000000   .000000000  00000-0  00000-0 0  9990
64 2 90010   0.0500 324.0000 0001000   0.0000 360.0000 1.00270000   11
65 HEO-01 (MOLNIYA)
66 1 90011U 25002A   25209.50000000   .000000000  00000-0  00000-0 0  9995
67 2 90011   63.4000  50.0000 7500000  270.0000  45.0000 2.00560000   13
68 HEO-02 (MOLNIYA)
```

```

69 1 90012U 25002B 25209.50000000 .00000000 00000-0 00000-0 0 9996
70 2 90012 63.4000 110.0000 7500000 270.0000 90.0000 2.00560000 14
71 HEO-03 (MOLNIYA)
72 1 90013U 25002C 25209.50000000 .00000000 00000-0 00000-0 0 9997
73 2 90013 63.4000 170.0000 7500000 270.0000 135.0000 2.00560000 15
74 HEO-04 (MOLNIYA)
75 1 90014U 25002D 25209.50000000 .00000000 00000-0 00000-0 0 9998
76 2 90014 63.4000 230.0000 7500000 270.0000 180.0000 2.00560000 16
77 HEO-05 (MOLNIYA)
78 1 90015U 25002E 25209.50000000 .00000000 00000-0 00000-0 0 9999
79 2 90015 63.4000 290.0000 7500000 270.0000 225.0000 2.00560000 17
80 LEO-01 (POLAR)
81 1 90016U 25003A 25209.50000000 .00000000 00000-0 00000-0 0 9990
82 2 90016 98.0000 120.0000 0010000 90.0000 20.0000 14.50000000 18
83 LEO-02 (POLAR)
84 1 90017U 25003B 25209.50000000 .00000000 00000-0 00000-0 0 9991
85 2 90017 98.0000 240.0000 0010000 90.0000 40.0000 14.50000000 19
86 LEO-03
87 1 90018U 25003C 25209.50000000 .00000000 00000-0 00000-0 0 9992
88 2 90018 45.0000 80.0000 0010000 45.0000 60.0000 15.00000000 10
89 LEO-04
90 1 90019U 25003D 25209.50000000 .00000000 00000-0 00000-0 0 9993
91 2 90019 45.0000 200.0000 0010000 45.0000 80.0000 15.00000000 11
92 LEO-05
93 1 90020U 25003E 25209.50000000 .00000000 00000-0 00000-0 0 9994
94 2 90020 28.5000 10.0000 0010000 10.0000 100.0000 15.50000000 12
95 """
96 dummy_tle_path = "standard_tle.txt"
97 with open(dummy_tle_path, "w") as f:
98     f.write(tle_data)
99
100 orbital_elements, epochs = readtle(dummy_tle_path)
101 num_sats = len(orbital_elements)
102
103 print(f"Initializing standard simulation with {num_sats} satellites.")
104
105 sim_data = initializeStructures(
106     num_satellites=num_sats,
107     num_observatories=0,
108     num_red_satellites=0,
109     start_time=start_time
110 )
111
112 sim_data['satellites']['orbital_elements'] = orbital_elements
113 sim_data['satellites']['epochs'] = epochs
114
115 solar_system_ephemeris.set('jpl')
116
117 print(f"Propagating satellites to start time: {start_time.isoformat()} to set
118 initial state.")
119 sim_data = propagate_satellites(sim_data, start_time)
120
121 return sim_data

```

2.3 constants.py

```

1 # --- Global Constants for Array Indices ---
2 # These constants define column indices for numpy arrays, making the
3 # code more readable and preventing errors from using "magic numbers".
4
5 # -- Radii in Meters --
6 EARTH_RADIUS = 6378137.0
7 MOON_RADIUS = 1737400.0
8
9 # -- Detector Array Indices --
10 DETECTOR_APERTURE_IDX = 0 # Aperture size in meters
11 DETECTOR_PIXEL_SIZE_IDX = 1 # Pixel size in radians
12 DETECTOR_QE_IDX = 2 # Quantum efficiency as a fraction (0.0 to 1.0)
13 DETECTOR_PIXELS_IDX = 3 # Total number of pixels in the detector (count)
14 DETECTOR_SOLAR_EXCL_IDX = 4 # Solar exclusion angle in radians
15 DETECTOR_LUNAR_EXCL_IDX = 5 # Lunar exclusion angle in radians
16 DETECTOR_EARTH_EXCL_IDX = 6 # Earth exclusion angle (above the limb) in radians

```

```

17
18 # -- Orbital Elements Array Indices --
19 ORBITAL_A_IDX = 0           # Semi-major axis in meters
20 ORBITAL_E_IDX = 1           # Eccentricity (dimensionless)
21 ORBITAL_I_IDX = 2           # Inclination in radians
22 ORBITAL_RAAN_IDX = 3        # Right Ascension of the Ascending Node in radians
23 ORBITAL_ARGP_IDX = 4        # Argument of Perigee in radians
24 ORBITAL_M_IDX = 5           # Mean Anomaly in radians
25
26 # -- Pointing State Array Indices --
27 POINTING_COUNT_IDX = 0      # Number of points in the pointing grid
28 POINTING_PLACE_IDX = 1      # Current index in the pointing sequence

```

2.4 demo1.py

```

1 from datetime import datetime, timezone, timedelta
2 import plotly.graph_objects as go
3
4 from common import initialize_standard_simulation
5 from propagation import celestial_update, propagate_satellites
6 from plotting_3d import plot_3d_scatter
7
8 def demo1() -> go.Figure:
9     """
10     Runs a full demonstration of the simulation tools.
11
12     This function generates and returns a Plotly figure object but does not
13     display it. The caller is responsible for rendering the plot.
14
15     Returns:
16         The Plotly figure object for the satellite positions plot.
17     """
18     sim_start_time = datetime(2025, 7, 27, 22, 27, 0, tzinfo=timezone.utc)
19     sim_data = initialize_standard_simulation(sim_start_time)
20
21     # --- Celestial Body Updates ---
22     sim_data = celestial_update(sim_data, sim_start_time)
23     print("\n--- Celestial Positions at Start Time ---")
24     print(f"Time: {sim_start_time.isoformat()}")
25     print(sim_data['celestial']['position'])
26
27     # --- Satellite Propagation and Plotting ---
28     time_t1 = sim_start_time + timedelta(hours=1, minutes=30)
29     print(f"\n--- Propagating satellites to T1: {time_t1.isoformat()} ---")
30     sim_data = propagate_satellites(sim_data, time_t1)
31
32     fig = plot_3d_scatter(
33         positions=sim_data['satellites']['position'],
34         title=f"Satellite Positions at {time_t1.isoformat()}",
35         plot_time=time_t1,
36         marker_size=2,
37         trace_name='Satellites'
38     )
39     return fig

```

2.5 demo2.py

```

1 import numpy as np
2 from datetime import datetime, timezone, timedelta
3 import plotly.graph_objects as go
4
5 from common import initialize_standard_simulation
6 from propagation import celestial_update, propagate_satellites
7
8 def demo2() -> go.Figure:
9     """
10     Runs a demonstration plotting satellite and celestial positions.
11
12     This function generates and returns a Plotly figure object but does not
13     display it. The caller is responsible for rendering the plot.

```

```

14
15 Returns:
16     The Plotly figure object.
17     """
18     sim_start_time = datetime(2025, 7, 27, 22, 27, 0, tzinfo=timezone.utc)
19     sim_data = initialize_standard_simulation(sim_start_time)
20
21     # --- Satellite and Celestial Propagation ---
22     positions_t0 = sim_data['satellites']['position'].copy()
23     sim_data = celestial_update(sim_data, sim_start_time)
24     celestial_pos_t0 = sim_data['celestial']['position'].copy()
25
26     time_t1 = sim_start_time + timedelta(seconds=300)
27     sim_data = propagate_satellites(sim_data, time_t1)
28     positions_t1 = sim_data['satellites']['position'].copy()
29     sim_data = celestial_update(sim_data, time_t1)
30     celestial_pos_t1 = sim_data['celestial']['position'].copy()
31
32     # --- 3D Plotting of All Time Steps ---
33     print("\n--- Generating 3D plot for Demo 2 ---")
34     earth_radius = 6378137.0
35     vector_scale = 10 * earth_radius
36     fig = go.Figure()
37
38     fig.add_trace(go.Scatter3d(
39         x=positions_t0[:, 0], y=positions_t0[:, 1], z=positions_t0[:, 2],
40         mode='markers', marker=dict(size=5, color='blue'), name='Sats (T=0s)'
41     ))
42     fig.add_trace(go.Scatter3d(
43         x=positions_t1[:, 0], y=positions_t1[:, 1], z=positions_t1[:, 2],
44         mode='markers', marker=dict(size=5, color='red'), name='Sats (T=300s)'
45     ))
46
47     sun_vec_t0 = celestial_pos_t0[0] / np.linalg.norm(celestial_pos_t0[0]) *
48     vector_scale
49     moon_vec_t0 = celestial_pos_t0[1] / np.linalg.norm(celestial_pos_t0[1]) *
50     vector_scale
51     fig.add_trace(go.Scatter3d(x=[0, sun_vec_t0[0]], y=[0, sun_vec_t0[1]], z=[0,
52     sun_vec_t0[2]], mode='lines', line=dict(color='orange', width=4), name='Sun (T=0s)'
53     ))
54     fig.add_trace(go.Scatter3d(x=[0, moon_vec_t0[0]], y=[0, moon_vec_t0[1]], z=[0,
55     moon_vec_t0[2]], mode='lines', line=dict(color='gray', width=4), name='Moon (T=0s)'
56     ))
57
58     sun_vec_t1 = celestial_pos_t1[0] / np.linalg.norm(celestial_pos_t1[0]) *
59     vector_scale
60     moon_vec_t1 = celestial_pos_t1[1] / np.linalg.norm(celestial_pos_t1[1]) *
61     vector_scale
62     fig.add_trace(go.Scatter3d(x=[0, sun_vec_t1[0]], y=[0, sun_vec_t1[1]], z=[0,
63     sun_vec_t1[2]], mode='lines', line=dict(color='yellow', width=4), name='Sun (T=300s)'
64     ))
65     fig.add_trace(go.Scatter3d(x=[0, moon_vec_t1[0]], y=[0, moon_vec_t1[1]], z=[0,
66     moon_vec_t1[2]], mode='lines', line=dict(color='lightgray', width=4), name='Moon (T
67     =300s)'
68     ))
69
70     u_sphere = np.linspace(0, 2 * np.pi, 100)
71     v_sphere = np.linspace(0, np.pi, 100)
72     x_earth = earth_radius * np.outer(np.cos(u_sphere), np.sin(v_sphere))
73     y_earth = earth_radius * np.outer(np.sin(u_sphere), np.sin(v_sphere))
74     z_earth = earth_radius * np.outer(np.ones(np.size(u_sphere)), np.cos(v_sphere))
75     fig.add_trace(go.Surface(x=x_earth, y=y_earth, z=z_earth, colorscale='Blues',
76     showscale=False, opacity=0.5, name='Earth'))
77
78     fig.update_layout(
79         title=f"Satellite and Celestial Positions at 0 and 300 seconds",
80         scene=dict(xaxis_title='X (m)', yaxis_title='Y (m)', zaxis_title='Z (m)',
81         aspectmode='data'),
82         margin=dict(r=20, b=10, l=10, t=40)
83     )
84
85     return fig

```

2.6 demo3.py

```
1 import numpy as np
2 from datetime import datetime, timezone, timedelta
3 import plotly.graph_objects as go
4
5 from propagation import readtle, propagate_satellites
6 from simulation import initializeStructures
7
8 def demo3() -> go.Figure:
9     """
10     Runs a demonstration plotting a single LEO satellite trajectory.
11
12     This function generates and returns a Plotly figure object but does not
13     display it. The caller is responsible for rendering the plot.
14
15     Returns:
16         The Plotly figure object.
17     """
18     sim_start_time = datetime(2025, 7, 27, 22, 27, 0, tzinfo=timezone.utc)
19
20     tle_data = '''LEO-TRAJECTORY
21 1 90201U 25005A   25210.50000000   .00000000   00000-0   00000-0 0   9991
22 2 90201   51.6000   10.0000 0010000   90.0000   20.0000 15.50000000    11
23 '''
24     dummy_tle_path = "dummy_tle3.txt"
25     with open(dummy_tle_path, "w") as f:
26         f.write(tle_data)
27
28     orbital_elements_from_tle, epochs_from_tle = readtle(dummy_tle_path)
29     num_sats = len(orbital_elements_from_tle)
30
31     print(f"\n--- Starting Demo 3 ---")
32     print(f"Initializing structures for {num_sats} LEO satellite.")
33
34     sim_data = initializeStructures(
35         num_satellites=num_sats,
36         num_observatories=0,
37         num_red_satellites=0,
38         start_time=sim_start_time
39     )
40     sim_data['satellites']['orbital_elements'] = orbital_elements_from_tle
41     sim_data['satellites']['epochs'] = epochs_from_tle
42
43     positions_over_time = []
44     time_steps = np.arange(0, 91, 10)
45
46     for minutes in time_steps:
47         prop_time = sim_start_time + timedelta(minutes=int(minutes))
48         sim_data = propagate_satellites(sim_data, prop_time)
49         positions_over_time.append(sim_data['satellites']['position'][0])
50
51     positions_array = np.array(positions_over_time)
52
53     print("\n--- Generating 3D plot for Demo 3 ---")
54     earth_radius = 6378137.0
55     fig = go.Figure()
56
57     fig.add_trace(go.Scatter3d(
58         x=positions_array[:, 0], y=positions_array[:, 1], z=positions_array[:, 2],
59         mode='lines', line=dict(color='red', width=4), name='Trajectory'
60     ))
61
62     fig.add_trace(go.Scatter3d(
63         x=positions_array[:, 0], y=positions_array[:, 1], z=positions_array[:, 2],
64         mode='markers', marker=dict(size=5, color='blue'),
65         text=[f'T={t} min' for t in time_steps], hoverinfo='text', name='Time Steps'
66     ))
67
68     u_sphere = np.linspace(0, 2 * np.pi, 100)
69     v_sphere = np.linspace(0, np.pi, 100)
70     x_earth = earth_radius * np.outer(np.cos(u_sphere), np.sin(v_sphere))
71     y_earth = earth_radius * np.outer(np.sin(u_sphere), np.sin(v_sphere))
```

```

72 z_earth = earth_radius * np.outer(np.ones(np.size(u_sphere)), np.cos(v_sphere))
73 fig.add_trace(go.Surface(x=x_earth, y=y_earth, z=z_earth, colorscale='Blues',
74                          showscale=False, opacity=0.5, name='Earth'))
75
76 fig.update_layout(
77     title=f"Single LEO Satellite Trajectory over 90 Minutes",
78     scene=dict(xaxis_title='X (m)', yaxis_title='Y (m)', zaxis_title='Z (m)',
79               aspectmode='data'),
80     margin=dict(r=20, b=10, l=10, t=40),
81     legend_title_text='Trace'
82 )
83
84 return fig

```

2.7 demo4.py

```

1 import numpy as np
2 from datetime import datetime, timezone, timedelta
3 import plotly.graph_objects as go
4
5 from propagation import readtle, propagate_satellites
6 from simulation import initializeStructures
7
8 def demo4() -> go.Figure:
9     """
10     Runs a demonstration plotting a single GEO satellite trajectory.
11
12     This function generates and returns a Plotly figure object but does not
13     display it. The caller is responsible for rendering the plot.
14
15     Returns:
16         The Plotly figure object.
17     """
18     sim_start_time = datetime(2025, 7, 27, 22, 27, 0, tzinfo=timezone.utc)
19
20     tle_data = '''GEO-TRAJECTORY
21 1 90301U 25006A   25210.50000000   .00000000   00000-0   00000-0 0   9991
22 2 90301   0.0500  45.0000 0001000   90.0000   20.0000   1.00270000    11
23 '''
24
25     dummy_tle_path = "dummy_tle4.txt"
26     with open(dummy_tle_path, "w") as f:
27         f.write(tle_data)
28
29     orbital_elements_from_tle, epochs_from_tle = readtle(dummy_tle_path)
30     num_sats = len(orbital_elements_from_tle)
31
32     print(f"\n--- Starting Demo 4 ---")
33     print(f"Initializing structures for {num_sats} GEO satellite.")
34
35     sim_data = initializeStructures(
36         num_satellites=num_sats,
37         num_observatories=0,
38         num_red_satellites=0,
39         start_time=sim_start_time
40     )
41     sim_data['satellites']['orbital_elements'] = orbital_elements_from_tle
42     sim_data['satellites']['epochs'] = epochs_from_tle
43
44     positions_over_time = []
45     time_steps = np.arange(0, 24, 1)
46
47     for hours in time_steps:
48         prop_time = sim_start_time + timedelta(hours=int(hours))
49         sim_data = propagate_satellites(sim_data, prop_time)
50         positions_over_time.append(sim_data['satellites']['position'][0])
51
52     positions_array = np.array(positions_over_time)
53
54     print("\n--- Generating 3D plot for Demo 4 ---")
55     earth_radius = 6378137.0
56     fig = go.Figure()

```



```

57 fig.add_trace(go.Scatter3d(
58     x=positions_array[:, 0], y=positions_array[:, 1], z=positions_array[:, 2],
59     mode='lines', line=dict(color='purple', width=4), name='Trajectory'
60 ))
61
62 fig.add_trace(go.Scatter3d(
63     x=positions_array[:, 0], y=positions_array[:, 1], z=positions_array[:, 2],
64     mode='markers', marker=dict(size=5, color='orange'),
65     text=[f'T={t} hr' for t in time_steps], hoverinfo='text', name='Time Steps'
66 ))
67
68 u_sphere = np.linspace(0, 2 * np.pi, 100)
69 v_sphere = np.linspace(0, np.pi, 100)
70 x_earth = earth_radius * np.outer(np.cos(u_sphere), np.sin(v_sphere))
71 y_earth = earth_radius * np.outer(np.sin(u_sphere), np.sin(v_sphere))
72 z_earth = earth_radius * np.outer(np.ones(np.size(u_sphere)), np.cos(v_sphere))
73 fig.add_trace(go.Surface(x=x_earth, y=y_earth, z=z_earth, colorscale='Blues',
74     showscale=False, opacity=0.5, name='Earth'))
75
76 fig.update_layout(
77     title=f"Single GEO Satellite Trajectory over 23 Hours",
78     scene=dict(xaxis_title='X (m)', yaxis_title='Y (m)', zaxis_title='Z (m)',
79     aspectmode='data'),
80     margin=dict(r=20, b=10, l=10, t=40),
81     legend_title_text='Trace'
82 )
83
84 return fig

```

2.8 demo_exclusion_debug_print.py

```

1 import numpy as np
2 from datetime import datetime, timezone
3
4 from common import initialize_standard_simulation
5 from propagation import celestial_update
6 from visibility import update_visibility_table
7 from simulation import DETECTOR_SOLAR_EXCL_IDX, DETECTOR_LUNAR_EXCL_IDX,
8     DETECTOR_EARTH_EXCL_IDX
9
10 def demo_exclusion_debug_print():
11     """
12     Demonstrates the debug printing feature of the exclusion function.
13
14     This function runs the exclusion check for the first satellite against
15     the first 100 fixed points and prints the detailed debug output for
16     each check, as enabled by the 'print_debug_for_sat' parameter.
17     """
18     print("\n--- Starting Demo: Exclusion Debug Print ---")
19     sim_start_time = datetime(2025, 8, 1, 12, 0, 0, tzinfo=timezone.utc)
20     sim_data = initialize_standard_simulation(sim_start_time)
21
22     sim_data['satellites']['detector'][:, DETECTOR_SOLAR_EXCL_IDX] = np.deg2rad(30)
23     sim_data['satellites']['detector'][:, DETECTOR_LUNAR_EXCL_IDX] = np.deg2rad(15)
24     sim_data['satellites']['detector'][:, DETECTOR_EARTH_EXCL_IDX] = np.deg2rad(10)
25
26     sim_data = celestial_update(sim_data, sim_start_time)
27
28     print("\n--- Generating exclusion table for Satellite 0 vs First 100 Fixed Points (
29     with debug print) ---")
30     original_fixed_points = sim_data['fixedpoints']['position']
31     sim_data['fixedpoints']['position'] = original_fixed_points[:100]
32
33     update_visibility_table(sim_data, print_debug_for_sat=0)
34
35     sim_data['fixedpoints']['position'] = original_fixed_points
36     print("\n--- Debug Print Demo Complete ---")

```

2.9 demo_exclusion_table.py

```

1 import numpy as np
2 from datetime import datetime, timezone
3 import plotly.graph_objects as go
4
5 from common import initialize_standard_simulation
6 from propagation import celestial_update
7 from visibility import update_visibility_table
8 from simulation import DETECTOR_SOLAR_EXCL_IDX, DETECTOR_LUNAR_EXCL_IDX,
   DETECTOR_EARTH_EXCL_IDX
9
10 def demo_exclusion_table() -> go.Figure:
11     """
12     Demonstrates the creation and visualization of the exclusion table.
13
14     This function generates and returns a Plotly figure object but does not
15     display it. The caller is responsible for rendering the plot.
16
17     Returns:
18         The Plotly figure object.
19     """
20     print("\n-- Starting Demo: Exclusion Table Visualization ---")
21     sim_start_time = datetime(2025, 8, 1, 12, 0, 0, tzinfo=timezone.utc)
22     sim_data = initialize_standard_simulation(sim_start_time)
23
24     sim_data['satellites']['detector'][:, DETECTOR_SOLAR_EXCL_IDX] = np.deg2rad(30)
25     sim_data['satellites']['detector'][:, DETECTOR_LUNAR_EXCL_IDX] = np.deg2rad(30)
26     sim_data['satellites']['detector'][:, DETECTOR_EARTH_EXCL_IDX] = np.deg2rad(10)
27
28     print("Calculating celestial body positions...")
29     sim_data = celestial_update(sim_data, sim_start_time)
30
31     print("Generating exclusion table (this may take a moment)...")
32     original_fixed_points = sim_data['fixedpoints']['position']
33     sim_data['fixedpoints']['position'] = original_fixed_points[:200]
34
35     update_visibility_table(sim_data)
36     exclusion_matrix = sim_data['fixedpoints']['visibility']
37
38     sim_data['fixedpoints']['position'] = original_fixed_points
39
40     print("Exclusion table generated.")
41
42     print("Displaying results as a heatmap...")
43     fig = go.Figure(data=go.Heatmap(
44         z=exclusion_matrix.T,
45         colorscale=[[0, 'red'], [1, 'green']],
46         showscale=False
47     ))
48
49     fig.update_layout(
50         title='Satellite vs. Fixed Point Exclusion Status (0=Clear, 1=Excluded)',
51         xaxis_title='Fixed Point Index',
52         yaxis_title='Satellite Index',
53         yaxis=dict(autorange='reversed')
54     )
55
56     return fig

```

2.10 demo_fixedpoints.py

```

1 from datetime import datetime, timezone
2 import plotly.graph_objects as go
3
4 from simulation import initializeStructures
5 from plotting_3d import plot_3d_scatter
6
7 def demo_fixedpoints() -> go.Figure:
8     """
9     Demonstrates the fixedpoints data structure by plotting it in 3D.
10
11     This function generates and returns a Plotly figure object but does not
12     display it. The caller is responsible for rendering the plot.

```

```

13
14 Returns:
15     The Plotly figure object.
16     """
17 print("\n--- Starting Demo Fixedpoints ---")
18 sim_start_time = datetime(2025, 7, 27, 22, 27, 0, tzinfo=timezone.utc)
19
20 sim_data = initializeStructures(
21     num_satellites=0,
22     num_observatories=0,
23     num_red_satellites=0,
24     start_time=sim_start_time
25 )
26
27 fixed_positions = sim_data['fixedpoints']['position']
28
29 print(f"Plotting {len(fixed_positions)} fixed points.")
30
31 fig = plot_3d_scatter(
32     positions=fixed_positions,
33     title="Fixed Points Distribution",
34     plot_time=sim_start_time,
35     labels=[f"Point {i}" for i in range(len(fixed_positions))],
36     marker_size=1,
37     trace_name='Fixed Points'
38 )
39 return fig

```

2.11 demo_lambertian.py

```

1 import numpy as np
2 import plotly.graph_objects as go
3 from lambertian import lambertiansphere
4
5 def demo_lambertian():
6     """
7     Runs a demonstration of the lambertiansphere function,
8     including example calculations and a plot.
9     """
10    SATELLITE_ALBEDO = 0.2
11    SATELLITE_RADIUS = 1.5
12
13    print(f"--- Simulating a sphere with Albedo={SATELLITE_ALBEDO} and Radius={SATELLITE_RADIUS}m ---\n")
14
15    print("--- Example 1: Full Illumination ---")
16    vec_sun_1 = np.array([1, 0, 0])
17    vec_obs_1 = np.array([1, 0, 0])
18    brightness_1 = lambertiansphere(vec_sun_1, vec_obs_1, SATELLITE_ALBEDO, SATELLITE_RADIUS)
19    angle_1 = np.rad2deg(np.arccos(np.dot(vec_sun_1, vec_obs_1)))
20    print(f"Phase Angle: {angle_1:.2f} degrees")
21    print(f"Effective Brightness: {brightness_1:.4f} m^2\n")
22
23    print("--- Example 2: Half Illumination ---")
24    vec_sun_2 = np.array([1, 0, 0])
25    vec_obs_2 = np.array([0, 1, 0])
26    brightness_2 = lambertiansphere(vec_sun_2, vec_obs_2, SATELLITE_ALBEDO, SATELLITE_RADIUS)
27    angle_2 = np.rad2deg(np.arccos(np.dot(vec_sun_2/np.linalg.norm(vec_sun_2), vec_obs_2/np.linalg.norm(vec_obs_2))))
28    print(f"Phase Angle: {angle_2:.2f} degrees")
29    print(f"Effective Brightness: {brightness_2:.4f} m^2\n")
30
31    print("--- Example 3: No Illumination ---")
32    vec_sun_3 = np.array([1, 0, 0])
33    vec_obs_3 = np.array([-1, 0, 0])
34    brightness_3 = lambertiansphere(vec_sun_3, vec_obs_3, SATELLITE_ALBEDO, SATELLITE_RADIUS)
35    angle_3 = np.rad2deg(np.arccos(np.dot(vec_sun_3, vec_obs_3)))
36    print(f"Phase Angle: {angle_3:.2f} degrees")
37    print(f"Effective Brightness: {brightness_3:.4f} m^2\n")

```

```

38
39 print("\n--- Generating Plot Data ---")
40 angles_deg = np.linspace(0, 180, 200)
41 angles_rad = np.deg2rad(angles_deg)
42 brightness_values = []
43 plot_vec_light = np.array([1, 0, 0])
44 for angle in angles_rad:
45     plot_vec_obs = np.array([np.cos(angle), np.sin(angle), 0])
46     brightness = lambertiansphere(plot_vec_light, plot_vec_obs, SATELLITE_ALBEDO,
47     SATELLITE_RADIUS)
48     brightness_values.append(brightness)
49
50 fig = go.Figure()
51 fig.add_trace(go.Scatter(x=angles_deg, y=brightness_values, mode='lines', name='
Effective Brightness'))
52 title_text = f'Lambertian Sphere Brightness vs. Phase Angle<br><sup>Albedo={
SATELLITE_ALBEDO}, Radius={SATELLITE_RADIUS}m</sup>'
53 fig.update_layout(
54     title_text=title_text,
55     xaxis_title="Physical Phase Angle (degrees)",
56     yaxis_title="Effective Brightness (m^2)",
57     template="plotly_white",
58     xaxis=dict(range=[0, 180]),
59     yaxis=dict(range=[0, max(brightness_values) * 1.1])
60 )
61
62 print("\n--- Displaying Plot ---")
63 fig.show()
64
65 if __name__ == '__main__':
66     demo_lambertian()

```

2.12 demo_pointing_plot.py

```

1 from datetime import datetime, timezone
2 import plotly.graph_objects as go
3
4 from common import initialize_standard_simulation
5 from plotting_vectors import plot_pointing_vectors
6
7 def demo_pointing_plot() -> go.Figure:
8     """
9     Demonstrates the plot_pointing_vectors function.
10
11     This function generates and returns a Plotly figure object but does not
12     display it. The caller is responsible for rendering the plot.
13
14     Returns:
15         The Plotly figure object.
16     """
17     print("\n--- Starting Demo: Pointing Vector Plot ---")
18     sim_start_time = datetime(2025, 8, 1, 12, 0, 0, tzinfo=timezone.utc)
19     sim_data = initialize_standard_simulation(sim_start_time)
20
21     fig = plot_pointing_vectors(
22         data_struct=sim_data,
23         title="Satellite Positions with Radially Outward Pointing Vectors",
24         plot_time=sim_start_time
25     )
26     return fig

```

2.13 demo_pointing_sequence.py

```

1 import numpy as np
2 from datetime import datetime, timezone, timedelta
3 import plotly.graph_objects as go
4
5 from simulation import initializeStructures
6 from constants import POINTING_COUNT_IDX, POINTING_PLACE_IDX

```

```

7 from pointing import generate_pointing_sphere, update_satellite_pointing,
  pointing_place_update
8 from plotting_vectors import plot_pointing_vectors
9
10 def demo_pointing_sequence() -> go.Figure:
11     """
12     Demonstrates the satellite pointing sequence functionality.
13     """
14     print("\n--- Starting Demo: Pointing Sequence ---")
15     sim_start_time = datetime(2025, 8, 1, 12, 0, 0, tzinfo=timezone.utc)
16
17     # Initialize a simulation with 3 satellites
18     sim_data = initializeStructures(
19         num_satellites=3,
20         num_observatories=0,
21         num_red_satellites=0,
22         start_time=sim_start_time
23     )
24
25     # Generate pointing spheres
26     generate_pointing_sphere(sim_data, 10)
27     generate_pointing_sphere(sim_data, 20)
28
29     # Assign pointing counts to satellites
30     pointing_state = sim_data['satellites']['pointing_state']
31     pointing_state[0, POINTING_COUNT_IDX] = 10
32     pointing_state[1, POINTING_COUNT_IDX] = 20
33     # Satellite 2 will have a pointing_count of 0 and should not move
34
35     print("Initial pointing vectors:")
36     update_satellite_pointing(sim_data)
37     print(sim_data['satellites']['pointing'])
38
39     # --- Create a figure to animate ---
40     fig = go.Figure()
41
42     # Initial plot
43     vectors = sim_data['satellites']['pointing']
44     fig.add_trace(go.Scatter3d(
45         x=vectors[:, 0], y=vectors[:, 1], z=vectors[:, 2],
46         mode='markers', marker=dict(size=10, color=['red', 'green', 'blue']),
47         name='T=0'
48     ))
49
50     # Simulation loop
51     for t in range(1, 5):
52         print(f"\n--- Time Step {t} ---")
53         pointing_place_update(sim_data)
54         update_satellite_pointing(sim_data)
55         print("Pointing vectors:")
56         print(sim_data['satellites']['pointing'])
57
58         vectors = sim_data['satellites']['pointing']
59         fig.add_trace(go.Scatter3d(
60             x=vectors[:, 0], y=vectors[:, 1], z=vectors[:, 2],
61             mode='markers', marker=dict(size=10, color=['red', 'green', 'blue'], opacity
62 =0.5),
63             name=f'T={t}'
64         ))
65
66     # Add a unit sphere for reference
67     u = np.linspace(0, 2 * np.pi, 100)
68     v = np.linspace(0, np.pi, 100)
69     x = np.outer(np.cos(u), np.sin(v))
70     y = np.outer(np.sin(u), np.sin(v))
71     z = np.outer(np.ones(np.size(u)), np.cos(v))
72     fig.add_trace(go.Surface(x=x, y=y, z=z, colorscale='Blues', showscale=False, opacity
73 =0.1))
74
75     fig.update_layout(
76         title="Satellite Pointing Sequence",
77         scene=dict(
78             xaxis_title='X', yaxis_title='Y', zaxis_title='Z',

```

```

77         aspectmode='data'
78     )
79 )
80
81     return fig
82
83 if __name__ == '__main__':
84     fig = demo_pointing_sequence()
85     fig.show()

```

2.14 demo_pointing_vectors.py

```

1 import plotly.graph_objects as go
2 from pointing_vectors import pointing_vectors, plot_vectors_on_sphere
3
4 def demo_pointing_vectors() -> go.Figure:
5     """
6     Demonstrates the generation and plotting of pointing vectors.
7
8     Returns:
9         A Plotly figure object.
10    """
11    print("--- Running Pointing Vectors Demo ---")
12    num_vectors = 1000
13    print(f"Generating {num_vectors} pointing vectors...")
14    vectors = pointing_vectors(num_vectors)
15    print("Plotting vectors on a sphere...")
16    fig = plot_vectors_on_sphere(vectors, f"{num_vectors} Pointing Vectors")
17    return fig
18
19 if __name__ == '__main__':
20     fig = demo_pointing_vectors()
21     fig.show()

```

2.15 demo_sky_scan.py

```

1 import numpy as np
2 from datetime import datetime, timezone
3 import plotly.graph_objects as go
4
5 from simulation import initializeStructures, DETECTOR_SOLAR_EXCL_IDX,
6     DETECTOR_LUNAR_EXCL_IDX, DETECTOR_EARTH_EXCL_IDX
7 from propagation import readtle, propagate_satellites, celestial_update
8 from visibility import exclusion
9
10 def demo_sky_scan() -> go.Figure:
11     """
12     Performs a sky scan from a GEO satellite to map celestial exclusion zones.
13     """
14     print("\n--- Starting Demo: Sky Scan from GEO ---")
15     sim_start_time = datetime(2025, 8, 1, 12, 0, 0, tzinfo=timezone.utc)
16
17     # a. Initialize a simulation with a single GEO satellite
18     tle_data = """GEO-SCAN
19 1 90401U 25007A   25210.50000000   .00000000   00000-0   00000-0 0 9991
20 2 90401   0.0500  45.0000 0001000   90.0000   20.0000   1.00270000   11
21 """
22     dummy_tle_path = "dummy_tle_skyscan.txt"
23     with open(dummy_tle_path, "w") as f:
24         f.write(tle_data)
25
26     orbital_elements, epochs = readtle(dummy_tle_path)
27     sim_data = initializeStructures(
28         num_satellites=1,
29         num_observatories=0,
30         num_red_satellites=0,
31         start_time=sim_start_time
32     )
33     sim_data['satellites']['orbital_elements'] = orbital_elements
34     sim_data['satellites']['epochs'] = epochs

```

```

34
35 # Set some reasonable exclusion angles
36 sim_data['satellites']['detector'][:, DETECTOR_SOLAR_EXCL_IDX] = np.deg2rad(30)
37 sim_data['satellites']['detector'][:, DETECTOR_LUNAR_EXCL_IDX] = np.deg2rad(15)
38 sim_data['satellites']['detector'][:, DETECTOR_EARTH_EXCL_IDX] = np.deg2rad(10)
39
40 # b. Update celestial body positions
41 sim_data = propagate_satellites(sim_data, sim_start_time)
42 sim_data = celestial_update(sim_data, sim_start_time)
43 print("Initialized GEO satellite and celestial bodies.")
44
45 # c. Create a 2D grid of pointing vectors
46 declinations = np.linspace(-np.pi / 2, np.pi / 2, 50)
47 right_ascensions = np.linspace(0, 2 * np.pi, 100)
48
49 # d. & e. Iterate and store results
50 sky_map = np.zeros((len(declinations), len(right_ascensions)))
51
52 print("Scanning the sky for exclusion zones...")
53 for i, dec in enumerate(declinations):
54     for j, ra in enumerate(right_ascensions):
55         # Convert spherical to Cartesian coordinates for the pointing vector
56         x = np.cos(dec) * np.cos(ra)
57         y = np.cos(dec) * np.sin(ra)
58         z = np.sin(dec)
59         pointing_vector = np.array([x, y, z])
60
61         # Update the satellite's pointing vector
62         sim_data['satellites']['pointing'][0] = pointing_vector
63
64         # Call the exclusion function
65         is_clear = exclusion(sim_data, satellite_index=0)
66         sky_map[i, j] = is_clear
67
68 print("Sky scan complete.")
69
70 # f. Use Plotly to create a heatmap
71 fig = go.Figure(data=go.Heatmap(
72     z=sky_map,
73     x=np.rad2deg(right_ascensions),
74     y=np.rad2deg(declinations),
75     colorscale=[[0, 'red'], [1, 'lightgreen']],
76     showscale=False,
77     colorbar=dict(
78         title='Visibility',
79         tickvals=[0, 1],
80         ticktext=['Excluded', 'Clear']
81     )
82 ))
83
84 fig.update_layout(
85     title='Sky Exclusion Map from a GEO Satellite',
86     xaxis_title='Right Ascension (degrees)',
87     yaxis_title='Declination (degrees)',
88     yaxis=dict(scaleanchor="x", scaleratio=1)
89 )
90
91 return fig
92
93 if __name__ == '__main__':
94     fig = demo_sky_scan()
95     fig.show()

```

2.16 generate_log_spherical_points.py

```

1 import numpy as np
2 from datetime import datetime, timezone
3 from plotting_3d import plot_3d_scatter
4
5
6 def generate_log_spherical_points(
7     num_points: int,

```

```

8     inner_radius: float,
9     outer_radius: float,
10    object_size_m: float = 1.0,
11    seed: int = None
12 ) -> tuple[np.ndarray, np.ndarray]:
13     """
14     Generates 3D points with logarithmic radial and uniform angular distribution.
15
16     This function creates a point cloud where point distances from the origin are
17     logarithmically spaced. On any given spherical shell, points are distributed
18     uniformly using the Fibonacci lattice method. Each point is associated with a
19     specified object size.
20
21     Args:
22         num_points: The total number of points to generate.
23         inner_radius: The minimum distance from the origin (must be positive).
24         outer_radius: The maximum distance from the origin (must be >= inner_radius).
25         object_size_m: The size in meters to be associated with each point.
26                        Defaults to 1.0.
27         seed: An optional integer to seed the random number generator for
28              reproducible shuffling.
29
30     Returns:
31         A tuple containing:
32         - A NumPy array of shape (num_points, 3) for the Cartesian coordinates.
33         - A NumPy array of shape (num_points,) for the object size in meters.
34     """
35     # Input validation
36     if not isinstance(num_points, int) or num_points <= 0:
37         raise ValueError("num_points must be a positive integer.")
38     if not (isinstance(inner_radius, (int, float)) and inner_radius > 0):
39         raise ValueError("inner_radius must be a positive number.")
40     if not (isinstance(outer_radius, (int, float)) and outer_radius >= inner_radius):
41         raise ValueError("outer_radius must be a positive number and >= inner_radius.")
42
43     # --- 1. Generate uniformly distributed unit vectors (Fibonacci Lattice) ---
44     # Create an array of indices for each point
45     indices = np.arange(0, num_points, dtype=float) + 0.5
46
47     # Uniformly distribute points along the z-axis (cos(phi))
48     z = 1 - 2 * indices / num_points
49
50     # Calculate the radius in the xy-plane for each point
51     radius_xy = np.sqrt(1 - z**2)
52
53     # Calculate the azimuthal angle using the golden angle
54     golden_angle = np.pi * (3. - np.sqrt(5.))
55     theta = golden_angle * indices
56
57     # Convert to Cartesian coordinates for the unit sphere
58     x = radius_xy * np.cos(theta)
59     y = radius_xy * np.sin(theta)
60
61     # Stack into an (N, 3) array of unit vectors
62     unit_vectors = np.stack([x, y, z], axis=1)
63
64     # --- 2. Generate logarithmically spaced radii ---
65     # The start and stop parameters for logspace are exponents
66     start_exp = np.log10(inner_radius)
67     stop_exp = np.log10(outer_radius)
68     radii = np.logspace(start_exp, stop_exp, num_points)
69
70     # --- 3. Shuffle radii to decouple radius from spherical position ---
71     # This prevents the smallest radii from always mapping to points near the
72     # north pole, creating a more uniform visual distribution of densities.
73     rng = np.random.default_rng(seed)
74     rng.shuffle(radii)
75
76     # --- 4. Scale unit vectors by radii using broadcasting ---
77     # Reshape radii to (N, 1) to broadcast with (N, 3) unit_vectors
78     points = unit_vectors * radii[:, np.newaxis]
79
80     # --- 5. Create an array for the object sizes ---

```



```

81     sizes = np.full(num_points, object_size_m, dtype=float)
82
83     return points, sizes
84
85 if __name__ == '__main__':
86     # --- Demo of Point Generation and Visualization ---
87     # This provides a consistent demonstration entry point, matching vibevolts_demo.py
88     NUM_POINTS = 100
89     # Using the same radii as the main simulation for consistency
90     INNER_RADIUS = 2000000
91     OUTER_RADIUS = 84328000
92     OBJECT_SIZE = 1.0
93
94     print(f"Generating {NUM_POINTS} points from radius {INNER_RADIUS} to {OUTER_RADIUS}
95     {...}")
96     generated_points, _ = generate_log_spherical_points(
97         num_points=NUM_POINTS,
98         inner_radius=INNER_RADIUS,
99         outer_radius=OUTER_RADIUS,
100         object_size_m=OBJECT_SIZE,
101         seed=42
102     )
103     print("Point generation complete.\n")
104
105     # Use the unified visualization function
106     plot_3d_scatter(
107         positions=generated_points,
108         title="Fixed Points Distribution (from generate_log_spherical_points.py)",
109         plot_time=datetime.now(timezone.utc),
110         labels=[f"Point {i}" for i in range(len(generated_points))],
111         marker_size=1,
112         trace_name='Fixed Points'
113     )

```

2.17 generate_report.py

```

1 import plotly.graph_objects as go
2
3 from vibevolts_demo import (
4     demo1,
5     demo2,
6     demo3,
7     demo4,
8     demo_fixedpoints,
9     demo_exclusion_table,
10    demo_pointing_plot,
11 )
12
13 def generate_demo_html_report():
14     """
15     Runs all plotting demos and saves the output to a single HTML file.
16     """
17     print("Generating HTML report for all demos...")
18
19     # A list of all demo functions that produce plots
20     plotting_demos = [
21         ("Demo 1: Satellite Positions", demo1),
22         ("Demo 2: Satellite and Celestial Positions", demo2),
23         ("Demo 3: Single LEO Satellite Trajectory", demo3),
24         ("Demo 4: Single GEO Satellite Trajectory", demo4),
25         ("Demo Fixed Points", demo_fixedpoints),
26         ("Demo Exclusion Table", demo_exclusion_table),
27         ("Demo Pointing Plot", demo_pointing_plot),
28     ]
29
30     # Collect all the figure objects
31     figures = []
32     for name, demo_func in plotting_demos:
33         print(f"Running: {name}")
34         fig = demo_func(show_plot=False)
35         # Add a title to the figure layout if it doesn't have one
36         if fig.layout.title.text is None or fig.layout.title.text == '':

```

```

37         fig.update_layout(title_text=name)
38         figures.append(fig)
39
40     # Write all figures to a single HTML file
41     report_filename = "demo_plots.html"
42     print(f"Writing report to {report_filename}...")
43     with open(report_filename, 'w') as f:
44         f.write("<html><head><title>Vibevolts Demo Report</title></head><body>")
45         f.write("<h1>Vibevolts Demo Report</h1>")
46         # Include plotly.js from CDN
47         f.write("<script src='https://cdn.plot.ly/plotly-latest.min.js'></script>")
48
49         for i, fig in enumerate(figures):
50             # Convert figure to an HTML div
51             fig_html = fig.to_html(full_html=False, include_plotlyjs=False)
52             f.write(f"<h2>{fig.layout.title.text}</h2>")
53             f.write(fig_html)
54             if i < len(figures) - 1:
55                 f.write("<hr>") # Add a horizontal rule between plots
56
57         f.write("</body></html>")
58
59     print("Report generation complete.")
60
61 if __name__ == '__main__':
62     generate_demo_html_report()

```

2.18 lambertian.py

```

1 import numpy as np
2
3 def lambertiansphere(
4     vec_from_sphere_to_light: np.ndarray,
5     vec_from_sphere_to_observer: np.ndarray,
6     albedo: float,
7     radius: float
8 ) -> float:
9     """
10     Calculates the effective brightness of a
11     Lambertian sphere.
12
13     This function determines the apparent brightness of
14     a diffusely reflecting sphere based on the angle
15     between the light source and the observer, the
16     sphere's albedo (reflectivity), and its size.
17
18     Args:
19         vec_from_sphere_to_light: A 3-element NumPy
20             array representing the direction vector from
21             the sphere to the light source.
22         vec_from_sphere_to_observer: A 3-element NumPy
23             array representing the direction vector from
24             the sphere to the observer.
25         albedo: The fraction of incident light that is
26             reflected (0.0 to 1.0).
27         radius: The radius of the sphere in meters.
28
29     Returns:
30         The effective brightness cross-section in
31         square meters. This value is proportional to
32         the total light reflected towards the observer.
33     """
34     if not 0.0 <= albedo <= 1.0:
35         raise ValueError("Albedo must be between 0.0 and 1.0.")
36     if radius < 0:
37         raise ValueError("Radius cannot be negative.")
38
39     norm_light = np.linalg.norm(vec_from_sphere_to_light)
40     norm_observer = np.linalg.norm(vec_from_sphere_to_observer)
41
42     if norm_light == 0 or norm_observer == 0:
43         raise ValueError("Input vectors cannot have zero length.")

```

```

44     unit_vec_light = vec_from_sphere_to_light / norm_light
45     unit_vec_observer = vec_from_sphere_to_observer / norm_observer
46
47     cos_alpha = np.dot(unit_vec_light, unit_vec_observer)
48     cos_alpha = np.clip(cos_alpha, -1.0, 1.0)
49     alpha = np.arccos(cos_alpha)
50
51     term1 = np.sin(alpha)
52     term2 = (np.pi - alpha) * np.cos(alpha)
53     phase_function_value = (2 / (3 * np.pi)) * (term1 + term2)
54
55     cross_sectional_area = np.pi * (radius ** 2)
56
57     effective_brightness = (
58         albedo *
59         cross_sectional_area *
60         phase_function_value
61     )
62
63     return effective_brightness
64

```

2.19 plotting_3d.py

```

1  import numpy as np
2  from datetime import datetime
3  from typing import List, Optional
4
5  import plotly.graph_objects as go
6  from astropy.time import Time
7  from astropy.coordinates import GCRS, EarthLocation
8  import astropy.units as u
9
10 EARTH_RADIUS = 6378137.0
11
12 def plot_3d_scatter(
13     positions: np.ndarray,
14     title: str,
15     plot_time: datetime,
16     labels: Optional[List[str]] = None,
17     marker_size: int = 1,
18     trace_name: str = 'Points'
19 ) -> go.Figure:
20     """
21     Creates a 3D plot of object positions.
22     """
23     if positions.ndim != 2 or positions.shape[1] != 3:
24         raise ValueError("positions array must have shape (n, 3).")
25
26     fig = go.Figure()
27
28     fig.add_trace(go.Scatter3d(
29         x=positions[:, 0], y=positions[:, 1], z=positions[:, 2],
30         mode='markers',
31         marker=dict(
32             size=marker_size,
33             color=np.arange(len(positions)),
34             colorscale='Viridis',
35             opacity=0.8
36         ),
37         text=labels,
38         hoverinfo='text' if labels else 'none',
39         name=trace_name
40     ))
41
42     u_sphere = np.linspace(0, 2 * np.pi, 100)
43     v_sphere = np.linspace(0, np.pi, 100)
44     x_earth = EARTH_RADIUS * np.outer(np.cos(u_sphere), np.sin(v_sphere))
45     y_earth = EARTH_RADIUS * np.outer(np.sin(u_sphere), np.sin(v_sphere))
46     z_earth = EARTH_RADIUS * np.outer(np.ones(np.size(u_sphere)), np.cos(v_sphere))
47     fig.add_trace(go.Surface(
48         x=x_earth, y=y_earth, z=z_earth,

```

```

49     colorscale='Blues', showscale=False, opacity=0.5, name='Earth'
50 ))
51
52 theta = np.linspace(0, 2 * np.pi, 100)
53 x_eq = EARTH_RADIUS * np.cos(theta)
54 y_eq = EARTH_RADIUS * np.sin(theta)
55 z_eq = np.zeros_like(theta)
56 fig.add_trace(go.Scatter3d(
57     x=x_eq, y=y_eq, z=z_eq, mode='lines',
58     line=dict(color='green', width=3), name='Equator'
59 ))
60
61 fig.add_trace(go.Scatter3d(
62     x=[0], y=[0], z=[EARTH_RADIUS * 1.1], mode='text',
63     text=['N'], textfont=dict(size=15, color='red'), name='North Pole'
64 ))
65
66 lat_es, lon_es = 33.92 * u.deg, -118.42 * u.deg
67 el_segundo_loc = EarthLocation.from_geodetic(lon=lon_es, lat=lat_es)
68 itr_coords = el_segundo_loc.get_itrs(obstime=Time(plot_time))
69 gcrs_coords = itr_coords.transform_to(GCRS(obstime=Time(plot_time)))
70 es_pos = gcrs_coords.cartesian.xyz.to(u.m).value * 1.05
71 fig.add_trace(go.Scatter3d(
72     x=[es_pos[0]], y=[es_pos[1]], z=[es_pos[2]], mode='text',
73     text=['ES'], textfont=dict(size=15, color='yellow'), name='El Segundo'
74 ))
75
76 fig.update_layout(
77     title=title,
78     scene=dict(
79         xaxis_title='X (m)', yaxis_title='Y (m)', zaxis_title='Z (m)',
80         aspectmode='data'
81     ),
82     margin=dict(r=20, b=10, l=10, t=40),
83     legend_title_text='Objects'
84 )
85
86 return fig

```

2.20 plotting_vectors.py

```

1 import numpy as np
2 from datetime import datetime
3 from typing import Dict, Any
4
5 import plotly.graph_objects as go
6
7 EARTH_RADIUS = 6378137.0
8
9 def plot_pointing_vectors(
10     data_struct: Dict[str, Any],
11     title: str,
12     plot_time: datetime
13 ) -> go.Figure:
14     """
15     Creates a 3D plot of satellites with pointing vectors.
16     """
17     sat_positions = data_struct['satellites']['position']
18     sat_pointing = data_struct['satellites']['pointing']
19     num_sats = data_struct['counts']['satellites']
20
21     if num_sats == 0:
22         print("No satellites to plot.")
23         return go.Figure()
24
25     fig = go.Figure()
26
27     fig.add_trace(go.Scatter3d(
28         x=sat_positions[:, 0], y=sat_positions[:, 1], z=sat_positions[:, 2],
29         mode='markers',
30         marker=dict(size=5, color='blue', opacity=0.8),
31         name='Satellites'

```

```

32 ))
33
34 vector_scale = 0.5 * EARTH_RADIUS
35 for i in range(num_sats):
36     start_point = sat_positions[i]
37     pointing_vec = sat_pointing[i]
38     norm = np.linalg.norm(pointing_vec)
39     unit_vec = pointing_vec / norm if norm > 0 else np.array([0, 0, 0])
40     end_point = start_point + unit_vec * vector_scale
41     fig.add_trace(go.Scatter3d(
42         x=[start_point[0], end_point[0]],
43         y=[start_point[1], end_point[1]],
44         z=[start_point[2], end_point[2]],
45         mode='lines',
46         line=dict(color='red', width=3),
47         showlegend=False
48     ))
49
50 u_sphere = np.linspace(0, 2 * np.pi, 100)
51 v_sphere = np.linspace(0, np.pi, 100)
52 x_earth = EARTH_RADIUS * np.outer(np.cos(u_sphere), np.sin(v_sphere))
53 y_earth = EARTH_RADIUS * np.outer(np.sin(u_sphere), np.sin(v_sphere))
54 z_earth = EARTH_RADIUS * np.outer(np.ones(np.size(u_sphere)), np.cos(v_sphere))
55 fig.add_trace(go.Surface(
56     x=x_earth, y=y_earth, z=z_earth,
57     colorscale='Blues', showscale=False, opacity=0.5, name='Earth'
58 ))
59
60 fig.update_layout(
61     title=title,
62     scene=dict(
63         xaxis_title='X (m)', yaxis_title='Y (m)', zaxis_title='Z (m)',
64         aspectmode='data'
65     ),
66     margin=dict(r=20, b=10, l=10, t=40)
67 )
68
69 return fig

```

2.21 pointing.py

```

1 import numpy as np
2 from typing import Dict, Any
3
4 from constants import POINTING_COUNT_IDX, POINTING_PLACE_IDX
5 from pointing_vectors import pointing_vectors
6
7 def pointing_place_update(data_struct: Dict[str, Any]) -> None:
8     """
9     Increments the pointing place for all satellites, wrapping around if necessary.
10    """
11    pointing_state = data_struct['satellites']['pointing_state']
12    pointing_counts = pointing_state[:, POINTING_COUNT_IDX]
13
14    # Increment pointing place
15    pointing_state[:, POINTING_PLACE_IDX] += 1
16
17    # Wrap around where place >= count
18    wrap_around_indices = np.where(pointing_state[:, POINTING_PLACE_IDX] >=
19    pointing_counts)
20    pointing_state[wrap_around_indices, POINTING_PLACE_IDX] = 0
21
22 def jerk(data_struct: Dict[str, Any], satellite_number: int) -> Dict[str, Any]:
23     """
24     Moves the pointing vector of a specific satellite by 0.3 radians in a
25     random direction.
26
27     This function applies a random rotation to the satellite's pointing vector
28     using a simplified version of Rodrigues' rotation formula.
29
30     Args:
31         data_struct: The main simulation data dictionary.

```

```

31     satellite_number: The index of the satellite to modify.
32
33     Returns:
34         The modified data_struct with the updated pointing vector.
35     """
36     p = data_struct['satellites']['pointing'][satellite_number]
37     p_norm = p / np.linalg.norm(p)
38
39     while True:
40         r = np.random.rand(3) - 0.5
41         k = np.cross(p_norm, r)
42         norm_k = np.linalg.norm(k)
43         if norm_k > 1e-9:
44             k_hat = k / norm_k
45             break
46
47     theta = 0.3
48     cos_theta = np.cos(theta)
49     sin_theta = np.sin(theta)
50
51     p_new = p_norm * cos_theta + np.cross(k_hat, p_norm) * sin_theta
52
53     data_struct['satellites']['pointing'][satellite_number] = p_new / np.linalg.norm(
54         p_new)
55
56     return data_struct
57
58 def generate_pointing_sphere(data_struct: Dict[str, Any], n_points: int) -> None:
59     """
60     Generates a pointing sphere with n_points and stores it in the data_struct.
61     If a sphere with the same number of points already exists, this function does
62     nothing.
63     """
64     if n_points not in data_struct['pointing_spheres']:
65         print(f"Generating pointing sphere with {n_points} points...")
66         data_struct['pointing_spheres'][n_points] = pointing_vectors(n_points)
67
68 def update_satellite_pointing(data_struct: Dict[str, Any]) -> None:
69     """
70     Updates the pointing vector for each satellite based on its pointing state.
71     """
72     num_sats = data_struct['counts']['satellites']
73     if num_sats == 0:
74         return
75
76     pointing_state = data_struct['satellites']['pointing_state']
77     pointing_vectors_all = data_struct['satellites']['pointing']
78
79     for i in range(num_sats):
80         count = pointing_state[i, POINTING_COUNT_IDX]
81         place = pointing_state[i, POINTING_PLACE_IDX]
82
83         if count > 0:
84             if count not in data_struct['pointing_spheres']:
85                 raise ValueError(f"Pointing sphere for {count} points not generated.")
86
87             grid = data_struct['pointing_spheres'][count]
88             pointing_vectors_all[i] = grid[place]
89
90 def find_and_jerk_blind_satellites(data_struct: Dict[str, Any]) -> Dict[str, Any]:
91     """
92     Finds satellites with no visibility and applies the 'jerk' function to them.
93
94     This function iterates through the visibility table. If any satellite (column)
95     has no visible fixed points (i.e., the column sum is 0), the 'jerk'
96     function is called to randomly adjust its pointing vector.
97
98     Args:
99         data_struct: The main simulation data dictionary.
100
101     Returns:
102         The modified data_struct.
103     """

```

```

102 visibility_table = data_struct['fixedpoints']['visibility']
103
104 column_sums = np.sum(visibility_table, axis=0)
105 blind_satellite_indices = np.where(column_sums == 0)[0]
106
107 for sat_index in blind_satellite_indices:
108     print(f"Satellite {sat_index} has no visible points. Applying jerk.")
109     data_struct = jerk(data_struct, sat_index)
110
111 return data_struct

```

2.22 pointing_vectors.py

```

1 import numpy as np
2 import plotly.graph_objects as go
3
4
5 def pointing_vectors(n: int) -> np.ndarray:
6     """
7     Generates n equally spaced points on a unit sphere using the Fibonacci lattice
8     algorithm.
9
10    Args:
11        n: The number of points to generate.
12
13    Returns:
14        A NumPy array of shape (n, 3) for the Cartesian coordinates of the points.
15    """
16    if not isinstance(n, int) or n <= 0:
17        raise ValueError("n must be a positive integer.")
18
19    indices = np.arange(0, n, dtype=float) + 0.5
20    z = 1 - 2 * indices / n
21    radius_xy = np.sqrt(1 - z**2)
22    golden_angle = np.pi * (3. - np.sqrt(5.))
23    theta = golden_angle * indices
24    x = radius_xy * np.cos(theta)
25    y = radius_xy * np.sin(theta)
26
27    unit_vectors = np.stack([x, y, z], axis=1)
28    return unit_vectors
29
30 def plot_vectors_on_sphere(vectors: np.ndarray, title: str) -> go.Figure:
31     """
32     Creates a 3D plot of vectors on a sphere.
33
34    Args:
35        vectors: A NumPy array of shape (n, 3) representing the vectors.
36        title: The title of the plot.
37
38    Returns:
39        A Plotly figure object.
40    """
41    if vectors.ndim != 2 or vectors.shape[1] != 3:
42        raise ValueError("vectors array must have shape (n, 3).")
43
44    fig = go.Figure()
45
46    fig.add_trace(go.Scatter3d(
47        x=vectors[:, 0], y=vectors[:, 1], z=vectors[:, 2],
48        mode='markers',
49        marker=dict(
50            size=2,
51            color='red',
52            opacity=0.8
53        ),
54        name='Vectors'
55    ))
56
57    u_sphere = np.linspace(0, 2 * np.pi, 100)
58    v_sphere = np.linspace(0, np.pi, 100)
59    x_earth = 1.0 * np.outer(np.cos(u_sphere), np.sin(v_sphere))

```

```

59 y_earth = 1.0 * np.outer(np.sin(u_sphere), np.sin(v_sphere))
60 z_earth = 1.0 * np.outer(np.ones(np.size(u_sphere)), np.cos(v_sphere))
61 fig.add_trace(go.Surface(
62     x=x_earth, y=y_earth, z=z_earth,
63     colorscale='Blues', showscale=False, opacity=0.5, name='Sphere'
64 ))
65
66 fig.update_layout(
67     title=title,
68     scene=dict(
69         xaxis_title='X', yaxis_title='Y', zaxis_title='Z',
70         aspectmode='data'
71     ),
72     margin=dict(r=20, b=10, l=10, t=40),
73     legend_title_text='Objects'
74 )
75
76 return fig

```

2.23 propagation.py

```

1  import numpy as np
2  from datetime import datetime, timezone
3  from typing import Dict, Any, List, Tuple
4  from sgp4.api import Satrec
5  from astropy.time import Time
6  from astropy.coordinates import get_body, GCRS
7  import astropy.units as u
8
9  from constants import (
10     ORBITAL_A_IDX, ORBITAL_E_IDX, ORBITAL_I_IDX,
11     ORBITAL_RAN_IDX, ORBITAL_ARGP_IDX, ORBITAL_M_IDX
12 )
13
14 def celestial_update(data_struct: Dict[str, Any], time_date: datetime) -> Dict[str, Any]:
15     """
16     Calculates and updates the positions of the Sun and Moon.
17
18     This function uses the astropy library to get the precise GCRS coordinates
19     of the Sun and Moon for the given time, and updates the 'celestial' position
20     component in the main data structure. Velocities and accelerations are not
21     calculated and remain zero.
22
23     Args:
24         data_struct: The main simulation data dictionary from initializeStructures.
25         time_date: The timezone-aware datetime object (in UTC) for the calculation.
26
27     Returns:
28         The modified data_struct with updated celestial positions.
29     """
30     if time_date.tzinfo is None:
31         raise ValueError("time_date must be timezone-aware.")
32
33     # Convert the python datetime object to an astropy Time object
34     astro_time = Time(time_date)
35
36     # Get Sun position in the GCRS frame using the modern get_body function
37     sun_coords = get_body("sun", astro_time)
38     sun_gcrs = sun_coords.transform_to(GCRS(obstime=astro_time))
39
40     # Get Moon position in the GCRS frame using the modern get_body function
41     moon_coords = get_body("moon", astro_time)
42     moon_gcrs = moon_coords.transform_to(GCRS(obstime=astro_time))
43
44     # Update the celestial data arrays (index 0 for Sun, 1 for Moon)
45     # Position data is converted from astropy's representation to a simple
46     # numpy array in meters.
47     celestial_pos = data_struct['celestial']['position']
48     celestial_pos[0] = sun_gcrs.cartesian.xyz.to(u.m).value
49     celestial_pos[1] = moon_gcrs.cartesian.xyz.to(u.m).value
50

```



```

51     return data_struct
52
53 def readtle(tle_file_path: str) -> Tuple[np.ndarray, List[datetime]]:
54     """
55     Reads a TLE file and extracts orbital elements and epochs for each satellite.
56
57     Args:
58         tle_file_path: The path to the TLE file.
59
60     Returns:
61         A tuple containing:
62         - A NumPy array of orbital elements.
63         - A list of datetime objects representing the epoch for each satellite.
64     """
65     orbital_elements_list = []
66     epochs_list = []
67     with open(tle_file_path, 'r') as f:
68         lines = f.readlines()
69
70     for i in range(0, len(lines), 3):
71         line1 = lines[i+1].strip()
72         line2 = lines[i+2].strip()
73
74         satellite = Satrec.twoline2rv(line1, line2)
75
76         jd, fr = satellite.jdsatepoch, satellite.jdsatepochF
77         epoch_dt = Time(jd, fr, format='jd', scale='utc').to_datetime(timezone.utc)
78         epochs_list.append(epoch_dt)
79
80         a = satellite.a * satellite.radiusearthkm * 1000.0
81         e = satellite.ecco
82         inc = satellite.inclo
83         raan = satellite.nodeo
84         argp = satellite.argpo
85         M = satellite.mo
86
87         elements = np.zeros(6)
88         elements[ORBITAL_A_IDX] = a
89         elements[ORBITAL_E_IDX] = e
90         elements[ORBITAL_I_IDX] = inc
91         elements[ORBITAL_RAAN_IDX] = raan
92         elements[ORBITAL_ARGP_IDX] = argp
93         elements[ORBITAL_M_IDX] = M
94         orbital_elements_list.append(elements)
95
96     return np.array(orbital_elements_list, dtype=float), epochs_list
97
98 def propagate_satellites(data_struct: Dict[str, Any], time_date: datetime) -> Dict[str, Any]:
99     """
100     Updates satellite positions and pointing vectors based on their orbital elements.
101
102     This function propagates the orbits of all satellites ('satellites' and
103     'red_satellites') from their TLE epoch to the specified time_date
104     using Kepler's laws. After calculating the new position, it sets the
105     satellite's pointing vector to be radially outward from the Earth's center.
106
107     Args:
108         data_struct: The main simulation data dictionary.
109         time_date: The timezone-aware datetime object (in UTC) to propagate to.
110
111     Returns:
112         The modified data_struct with updated satellite positions and pointing vectors.
113     """
114     MU_EARTH = 3.986004418e14
115
116     time_date_timestamp = time_date.timestamp()
117
118     for sat_category in ['satellites', 'red_satellites']:
119         if data_struct['counts'][sat_category] == 0:
120             continue
121
122         elements = data_struct[sat_category]['orbital_elements']

```

```

123     epochs = data_struct[sat_category]['epochs']
124
125     epoch_timestamps = np.array([e.timestamp() for e in epochs])
126     delta_t_array = time_date_timestamp - epoch_timestamps
127
128     a = elements[:, ORBITAL_A_IDX]
129     e = elements[:, ORBITAL_E_IDX]
130     i = elements[:, ORBITAL_I_IDX]
131     raan = elements[:, ORBITAL_RAAN_IDX]
132     argp = elements[:, ORBITAL_ARGP_IDX]
133     MO = elements[:, ORBITAL_M_IDX]
134
135     n = np.sqrt(MU_EARTH / a**3)
136     M = (MO + n * delta_t_array) % (2 * np.pi)
137
138     E = M.copy()
139     for _ in range(10):
140         f_E = E - e * np.sin(E) - M
141         f_prime_E = 1 - e * np.cos(E)
142         f_prime_E[f_prime_E == 0] = 1e-10
143         E = E - f_E / f_prime_E
144
145     tan_nu_half = np.sqrt((1 + e) / (1 - e)) * np.tan(E / 2)
146     nu = 2 * np.arctan(tan_nu_half)
147
148     r = a * (1 - e * np.cos(E))
149
150     x_pqw = r * np.cos(nu)
151     y_pqw = r * np.sin(nu)
152
153     cos_raan = np.cos(raan)
154     sin_raan = np.sin(raan)
155     cos_argp = np.cos(argp)
156     sin_argp = np.sin(argp)
157     cos_i = np.cos(i)
158     sin_i = np.sin(i)
159
160     P_x = cos_argp * cos_raan - sin_argp * sin_raan * cos_i
161     P_y = cos_argp * sin_raan + sin_argp * cos_raan * cos_i
162     P_z = sin_argp * sin_i
163
164     Q_x = -sin_argp * cos_raan - cos_argp * sin_raan * cos_i
165     Q_y = -sin_argp * sin_raan + cos_argp * cos_raan * cos_i
166     Q_z = cos_argp * sin_i
167
168     x_gcrs = x_pqw * P_x + y_pqw * Q_x
169     y_gcrs = x_pqw * P_y + y_pqw * Q_y
170     z_gcrs = x_pqw * P_z + y_pqw * Q_z
171
172     positions = np.vstack((x_gcrs, y_gcrs, z_gcrs)).T
173     data_struct[sat_category]['position'] = positions
174
175     norms = np.linalg.norm(positions, axis=1)[: , np.newaxis]
176     norms[norms == 0] = 1.0
177     data_struct[sat_category]['pointing'] = positions / norms
178
179     return data_struct

```

2.24 radiometry_calcs.py

```

1 import math
2 import scipy.integrate
3 import scipy.constants as const
4 import numpy as np
5 import plotly.graph_objects as go
6
7 def mag(x: float) -> float:
8     """
9     Calculates a magnitude value from a linear ratio.
10    Uses the formula: magnitude = -2.5 * log10(ratio)
11    """
12    if x <= 0:

```

```

13         raise ValueError("Input for mag() must be positive.")
14     return -2.5 * math.log10(x)
15
16 def amag(x: float) -> float:
17     """
18     Calculates the linear ratio from a magnitude value.
19     This is the inverse of the mag() function.
20     Uses the formula: ratio = 10**(-0.4 * magnitude)
21     """
22     return 10**(-0.4 * x)
23
24 def _planck_law(wav_m: float, temp_k: float) -> float:
25     """
26     Helper function for Planck's law for spectral radiance.
27     Args:
28         wav_m: Wavelength in meters.
29         temp_k: Temperature in Kelvin.
30     Returns:
31         Spectral radiance in W / (m^2 * sr * m).
32     """
33     if wav_m <= 0:
34         return 0.0
35     exponent = (const.h * const.c) / (wav_m * const.k * temp_k)
36     if exponent > 700:
37         return 0.0
38     numerator = 2.0 * const.h * const.c**2
39     denominator = (wav_m**5) * (math.exp(1/exponent))
40     if denominator == 0:
41         return 0.0
42     return numerator / denominator
43
44 def blackbody_flux(
45     temperature: float,
46     lambda_short: float,
47     lambda_long: float
48 ) -> float:
49     """
50     Numerically computes the integrated spectral radiance of
51     a blackbody over a given wavelength band.
52
53     Requires the scipy library.
54
55     Args:
56         temperature: The temperature of the blackbody in Kelvin.
57         lambda_short: The short wavelength of the band in microns.
58         lambda_long: The long wavelength of the band in microns.
59
60     Returns:
61         The integrated spectral radiance in units of
62         Watts / (m^2 * steradian).
63     """
64     if lambda_short >= lambda_long:
65         raise ValueError("lambda_short must be less than lambda_long.")
66     lambda_short_m = lambda_short * 1e-6
67     lambda_long_m = lambda_long * 1e-6
68     integrated_radiance, _ = scipy.integrate.quad(
69         lambda wav: _planck_law(wav, temperature),
70         lambda_short_m,
71         lambda_long_m
72     )
73     return integrated_radiance
74
75 def stefan_boltzmann_law(temperature: float) -> float:
76     """
77     Calculates the total power radiated per unit area by a
78     blackbody using the Stefan-Boltzmann law.
79
80     Args:
81         temperature: The temperature of the blackbody in Kelvin.
82
83     Returns:
84         The total radiated power per unit area in W / m^2.
85     """

```

```

86     return const.sigma * (temperature**4)
87
88 def plot_blackbody_spectrum(temperature: float):
89     """
90     Plots the spectral radiance of a blackbody from
91     0.5 to 30 microns.
92
93     Args:
94         temperature: The temperature of the blackbody in Kelvin.
95     """
96     wavelengths_microns = np.geomspace(0.5, 30, 500)
97     wavelengths_m = wavelengths_microns * 1e-6
98     spectral_radiance = [_planck_law(w, temperature) * 1e-6 for w in wavelengths_m]
99     fig = go.Figure()
100     fig.add_trace(go.Scatter(
101         x=wavelengths_microns,
102         y=spectral_radiance,
103         mode='lines',
104         name=f'{temperature} K Blackbody'
105     ))
106     fig.update_layout(
107         title_text=f'Blackbody Spectrum ({temperature} K)',
108         xaxis_title="Wavelength (microns)",
109         yaxis_title="Spectral Radiance (W / m^2 / sr / micron)",
110         xaxis_type="log",
111         yaxis_type="log",
112         template="plotly_white"
113     )
114     fig.show()
115
116 def plot_blackbody_spectrum_visible_nir(temperature: float):
117     """
118     Plots the spectral radiance of a blackbody from
119     0.1 to 1 micron.
120
121     Args:
122         temperature: The temperature of the blackbody in Kelvin.
123     """
124     wavelengths_microns = np.linspace(0.1, 1.0, 500)
125     wavelengths_m = wavelengths_microns * 1e-6
126     spectral_radiance = [_planck_law(w, temperature) * 1e-6 for w in wavelengths_m]
127     fig = go.Figure()
128     fig.add_trace(go.Scatter(
129         x=wavelengths_microns,
130         y=spectral_radiance,
131         mode='lines',
132         name=f'{temperature} K Blackbody'
133     ))
134     fig.update_layout(
135         title_text=f'Blackbody Spectrum ({temperature} K) - Visible/NIR',
136         xaxis_title="Wavelength (microns)",
137         yaxis_title="Spectral Radiance (W / m^2 / sr / micron)",
138         template="plotly_white"
139     )
140     fig.show()

```

2.25 radiometry_data.py

```

1 import scipy.constants as const
2
3 # --- Physical Constants ---
4 AU_M = const.au # Astronomical Unit in meters
5 RSUN_M = 6.957e8 # Radius of the Sun in meters
6
7 # --- Radiometric Data ---
8
9 # Data is compiled for standard astronomical filters.
10 #
11 # Magnitudes are apparent magnitudes in the AB or Vega system.
12 # Sky brightness is for a dark site at new moon, in
13 # magnitudes per square arcsecond. For space-based IR
14 # telescopes, this is the zodiacal + telescope background.

```

```

15 # For ground-based IR, it's dominated by thermal emission.
16 # Wavelengths and bandwidths are in nanometers (nm).
17 # Zero point is the photon flux for a 0-magnitude
18 # object in photons per second per square meter.

```

```

19
20 FILTER_DATA = {
21     # Johnson-Cousins UBVRI Filters
22     'U': {
23         'sun': -26.03,
24         'sky': 22.0,
25         'central_wavelength': 365.0,
26         'bandwidth': 66.0,
27         'zero_point': 4.96e9,
28     },
29     'B': {
30         'sun': -26.13,
31         'sky': 22.7,
32         'central_wavelength': 445.0,
33         'bandwidth': 94.0,
34         'zero_point': 1.36e10,
35     },
36     'V': {
37         'sun': -26.78,
38         'sky': 21.9,
39         'central_wavelength': 551.0,
40         'bandwidth': 88.0,
41         'zero_point': 8.79e9,
42     },
43     'R': {
44         'sun': -27.11,
45         'sky': 21.0,
46         'central_wavelength': 658.0,
47         'bandwidth': 138.0,
48         'zero_point': 9.74e9,
49     },
50     'I': {
51         'sun': -27.47,
52         'sky': 20.0,
53         'central_wavelength': 806.0,
54         'bandwidth': 149.0,
55         'zero_point': 7.11e9,
56     },
57     # Near-Infrared JHK Filters
58     'J': {
59         'sun': -27.91,
60         'sky': 16.0,
61         'central_wavelength': 1235.0,
62         'bandwidth': 162.0,
63         'zero_point': 3.15e9,
64     },
65     'H': {
66         'sun': -28.25,
67         'sky': 14.0,
68         'central_wavelength': 1646.0,
69         'bandwidth': 251.0,
70         'zero_point': 2.35e9,
71     },
72     'K': {
73         'sun': -28.29,
74         'sky': 13.0,
75         'central_wavelength': 2159.0,
76         'bandwidth': 262.0,
77         'zero_point': 1.17e9,
78     },
79     # SDSS Filters
80     'g': {
81         'sun': -26.47,
82         'sky': 21.8,
83         'central_wavelength': 477.0,
84         'bandwidth': 137.9,
85         'zero_point': 1.582e10,
86     },
87     'r': {

```

```

88         'sun': -26.93,
89         'sky': 20.8,
90         'central_wavelength': 623.1,
91         'bandwidth': 138.2,
92         'zero_point': 1.214e10,
93     },
94     'i': {
95         'sun': -27.05,
96         'sky': 20.2,
97         'central_wavelength': 762.5,
98         'bandwidth': 153.5,
99         'zero_point': 1.103e10,
100     },
101     'z': {
102         'sun': -27.07,
103         'sky': 19.0,
104         'central_wavelength': 913.4,
105         'bandwidth': 140.9,
106         'zero_point': 8.455e9,
107     },
108     # Ground-Based Mid-IR Filters
109     'L': {
110         'sun': None,
111         'sky': 3.5,
112         'central_wavelength': 3500.0,
113         'bandwidth': 600.0,
114         'zero_point': 9.4e9,
115     },
116     'M': {
117         'sun': None,
118         'sky': 0.0,
119         'central_wavelength': 4800.0,
120         'bandwidth': 600.0,
121         'zero_point': 6.8e9,
122     },
123     'N': {
124         'sun': None,
125         'sky': -6.0,
126         'central_wavelength': 10200.0,
127         'bandwidth': 5000.0,
128         'zero_point': 2.7e10,
129     },
130     # JWST MIRI Filters
131     'F560W': { 'sun': None, 'sky': 24.4, 'central_wavelength': 5600.0, 'bandwidth':
132     1000.0, 'zero_point': 9.78e9 },
133     'F770W': { 'sun': None, 'sky': 24.4, 'central_wavelength': 7700.0, 'bandwidth':
134     1950.0, 'zero_point': 1.39e10 },
135     'F1000W': { 'sun': None, 'sky': 23.6, 'central_wavelength': 10000.0, 'bandwidth':
136     1800.0, 'zero_point': 9.86e9 },
137     'F1500W': { 'sun': None, 'sky': 21.9, 'central_wavelength': 15000.0, 'bandwidth':
138     2940.0, 'zero_point': 1.08e10 },
139     'F2550W': { 'sun': None, 'sky': 19.4, 'central_wavelength': 25500.0, 'bandwidth':
140     4050.0, 'zero_point': 8.70e9 }
141 }

```

2.26 simulation.py

```

1 import numpy as np
2 from datetime import datetime, timezone
3 from typing import Dict, Any
4 # Assuming generate_log_spherical_points is in the same directory or accessible
5 from generate_log_spherical_points import generate_log_spherical_points
6 from constants import *
7
8 from pointing import generate_pointing_sphere
9
10
11
12 def initializeStructures(
13     num_satellites: int,
14     num_observatories: int,
15     num_red_satellites: int,

```

```

16     start_time: datetime,
17     delta_time: float = 60.0
18 ) -> Dict[str, Any]:
19     """
20     Initializes categorized data structures for a space simulation.
21
22     This function creates distinct sets of data components for different categories
23     of entities. It includes kinematic data, orbital elements, sensor pointing
24     information, and detailed detector characteristics.
25
26     The 'orbital_elements' component contains the following columns:
27     - 0: Semi-major axis (meters)
28     - 1: Eccentricity
29     - 2: Inclination (radians)
30     - 3: Right Ascension of the Ascending Node (radians)
31     - 4: Argument of Perigee (radians)
32     - 5: Mean Anomaly (radians)
33
34     The 'detector' component contains the following columns:
35     - 0: Aperture size (meters)
36     - 1: Pixel size (radians)
37     - 2: Quantum Efficiency (fraction)
38     - 3: Pixel count (total number)
39     - 4: Solar exclusion angle (radians)
40     - 5: Lunar exclusion angle (radians)
41     - 6: Earth exclusion angle (radians, above the limb)
42
43     The 'fixedpoints' component is initialized with a set of 10,000 static points
44     in the GCRS frame, generated by 'generate_log_spherical_points'. These
45     points have a logarithmic radial distribution and are intended for various
46     fixed-point calculations or as a reference grid.
47
48     All kinematic data is for the Geocentric Celestial Reference System (GCRS).
49     Angles are relative to the International Celestial Reference System (ICRS).
50     Distances are in meters, angles in radians, and time in seconds.
51
52     Args:
53         num_satellites: The number of regular space-based sensors.
54         num_observatories: The number of ground-based sensors.
55         num_red_satellites: The number of special "red" satellites.
56         start_time: The starting time and date of the simulation. This must be a
57                     timezone-aware datetime object set to UTC.
58
59     Returns:
60         A dictionary representing the simulation state, containing categorized data.
61     """
62     # --- Input Validation ---
63     if not isinstance(num_satellites, int) or num_satellites < 0:
64         raise ValueError("num_satellites must be a non-negative integer.")
65     if not isinstance(num_observatories, int) or num_observatories < 0:
66         raise ValueError("num_observatories must be a non-negative integer.")
67     if not isinstance(num_red_satellites, int) or num_red_satellites < 0:
68         raise ValueError("num_red_satellites must be a non-negative integer.")
69     if not isinstance(start_time, datetime):
70         raise TypeError("start_time must be a datetime object.")
71     if start_time.tzinfo is None:
72         raise ValueError("start_time must be timezone-aware. Please set tzinfo.")
73
74     num_fixed_points = 100
75
76     # --- Main Data Structure ---
77     simulation_data: Dict[str, Any] = {
78         'start_time': start_time,
79         'delta_time': delta_time,
80         'counts': {
81             'celestial': 2, # Sun and Moon
82             'satellites': num_satellites,
83             'observatories': num_observatories,
84             'red_satellites': num_red_satellites,
85             'fixedpoints': num_fixed_points
86         },
87
88         'celestial': {

```

```

89         'position': np.zeros((2, 3), dtype=float),
90         'velocity': np.zeros((2, 3), dtype=float),
91         'acceleration': np.zeros((2, 3), dtype=float),
92     },
93
94     'satellites': {
95         'position': np.zeros((num_satellites, 3), dtype=float),
96         'velocity': np.zeros((num_satellites, 3), dtype=float),
97         'acceleration': np.zeros((num_satellites, 3), dtype=float),
98         'orbital_elements': np.zeros((num_satellites, 6), dtype=float),
99         'epochs': [], # List to store datetime epochs for each satellite
100         'pointing': np.zeros((num_satellites, 3), dtype=float),
101         'pointing_state': np.zeros((num_satellites, 2), dtype=int),
102         'detector': np.zeros((num_satellites, 7), dtype=float),
103     },
104
105     # FIX: Added the 'pointing_spheres' key, initialized as an empty dictionary.
106     'pointing_spheres': { } ,
107
108     'observatories': {
109         'position': np.zeros((num_observatories, 3), dtype=float),
110         'velocity': np.zeros((num_observatories, 3), dtype=float),
111         'acceleration': np.zeros((num_observatories, 3), dtype=float),
112         'detector': np.zeros((num_observatories, 7), dtype=float),
113     },
114
115     'red_satellites': {
116         'position': np.zeros((num_red_satellites, 3), dtype=float),
117         'velocity': np.zeros((num_red_satellites, 3), dtype=float),
118         'acceleration': np.zeros((num_red_satellites, 3), dtype=float),
119         'orbital_elements': np.zeros((num_red_satellites, 6), dtype=float),
120         'epochs': [],
121         'pointing': np.zeros((num_red_satellites, 3), dtype=float),
122         'detector': np.zeros((num_red_satellites, 7), dtype=float),
123     },
124
125     'fixedpoints': {
126         'position': generate_log_spherical_points(
127             num_points=num_fixed_points,
128             inner_radius=2000000,
129             # Outer radius is 2 * geostationary radius (42,164 km)
130             # This is an interpretation of "tice geodistances".
131             outer_radius=84328000
132         )[0], # Only get the positions, not the sizes
133         'visibility': np.zeros((num_fixed_points, num_satellites), dtype=int)
134     }
135 }
136
137 generate_pointing_sphere(simulation_data,100)
138
139 return simulation_data
140

```

2.27 visibility.py

```

1 import numpy as np
2 from typing import Dict, Any, Tuple, Optional
3
4 from constants import (
5     EARTH_RADIUS, MOON_RADIUS, DETECTOR_SOLAR_EXCL_IDX,
6     DETECTOR_LUNAR_EXCL_IDX, DETECTOR_EARTH_EXCL_IDX
7 )
8
9 def solarexclusion(data_struct: Dict[str, Any]) -> Tuple[np.ndarray, np.ndarray]:
10     """
11     Calculates solar exclusion for all satellites based on their pointing vectors.
12
13     This function operates in a vectorized manner on all satellites in the
14     'satellites' category. It computes the angle between each satellite's
15     pointing vector and the vector from the satellite to the Sun.
16
17     Args:

```



```

18     data_struct: The main simulation data dictionary.
19
20 Returns:
21     A tuple containing:
22     - exclusion_vector (np.ndarray): An array of the same length as the
23       number of satellites. An element is 1 if the satellite is within
24       the solar exclusion angle, 0 otherwise.
25     - angle_vector (np.ndarray): An array containing the calculated angle
26       in radians for each satellite.
27
28 """
29 num_sats = data_struct['counts']['satellites']
30 if num_sats == 0:
31     return np.array([]), np.array([])
32
33 sun_pos = data_struct['celestial']['position'][0]
34 sat_pos = data_struct['satellites']['position']
35 sat_pointing = data_struct['satellites']['pointing']
36 solar_exclusion_angles = data_struct['satellites']['detector'][:,
37     DETECTOR_SOLAR_EXCL_IDX]
38
39 vec_sat_to_sun = sun_pos - sat_pos
40
41 norm_sat_to_sun = np.linalg.norm(vec_sat_to_sun, axis=1)
42 norm_sat_pointing = np.linalg.norm(sat_pointing, axis=1)
43
44 valid_norms = (norm_sat_to_sun > 1e-9) & (norm_sat_pointing > 1e-9)
45
46 angle_vector = np.full(num_sats, np.pi)
47
48 if np.any(valid_norms):
49     dot_product = np.einsum('ij,ij->i', vec_sat_to_sun[valid_norms], sat_pointing[
50     valid_norms])
51     cos_angle = dot_product / (norm_sat_to_sun[valid_norms] * norm_sat_pointing[
52     valid_norms])
53     cos_angle = np.clip(cos_angle, -1.0, 1.0)
54     angle_vector[valid_norms] = np.arccos(cos_angle)
55
56 exclusion_vector = (angle_vector < solar_exclusion_angles).astype(int)
57
58 return exclusion_vector, angle_vector
59
60 def exclusion(
61     data_struct: Dict[str, Any],
62     satellite_index: int,
63     print_debug: bool = False
64 ) -> int:
65     """
66     Determines if a satellite's pointing vector is excluded by the Sun, Moon, or Earth.
67
68     Args:
69         data_struct: The main simulation data dictionary.
70         satellite_index: The index of the satellite to check.
71         print_debug: If True, prints detailed debug information for the calculation.
72
73     Returns:
74         0 if the satellite's view is excluded, 1 otherwise.
75     """
76     sat_pos = data_struct['satellites']['position'][satellite_index]
77     sat_pointing = data_struct['satellites']['pointing'][satellite_index]
78     sun_pos = data_struct['celestial']['position'][0]
79     moon_pos = data_struct['celestial']['position'][1]
80
81     detector_props = data_struct['satellites']['detector'][satellite_index]
82     solar_excl_angle = detector_props[DETECTOR_SOLAR_EXCL_IDX]
83     lunar_excl_angle = detector_props[DETECTOR_LUNAR_EXCL_IDX]
84     earth_excl_angle = detector_props[DETECTOR_EARTH_EXCL_IDX]
85
86     vec_to_sun = sun_pos - sat_pos
87     vec_to_moon = moon_pos - sat_pos
88     vec_to_earth = -sat_pos
89
90     dist_to_sun = np.linalg.norm(vec_to_sun)
91     dist_to_moon = np.linalg.norm(vec_to_moon)

```

```

88     dist_to_earth = np.linalg.norm(vec_to_earth)
89
90     u_vec_to_sun = vec_to_sun / dist_to_sun if dist_to_sun > 0 else np.array([0.,0.,0.])
91     u_vec_to_moon = vec_to_moon / dist_to_moon if dist_to_moon > 0 else np.array
92     ([0.,0.,0.])
93     u_vec_to_earth = vec_to_earth / dist_to_earth if dist_to_earth > 0 else np.array
94     ([0.,0.,0.])
95
96     norm_pointing = np.linalg.norm(sat_pointing)
97     u_sat_pointing = sat_pointing / norm_pointing if norm_pointing > 0 else np.array
98     ([0.,0.,0.])
99
100     sun_flag, moon_flag, earth_flag = False, False, False
101
102     cos_angle_sun = np.clip(np.dot(u_sat_pointing, u_vec_to_sun), -1.0, 1.0)
103     angle_sun = np.arccos(cos_angle_sun)
104     if angle_sun < solar_excl_angle:
105         sun_flag = True
106
107     cos_angle_moon = np.clip(np.dot(u_sat_pointing, u_vec_to_moon), -1.0, 1.0)
108     angle_moon = np.arccos(cos_angle_moon)
109     apparent_radius_moon = np.arctan(MOON_RADIUS / dist_to_moon) if dist_to_moon > 0
110     else 0
111     if (angle_moon - apparent_radius_moon) < lunar_excl_angle:
112         moon_flag = True
113
114     cos_angle_earth = np.clip(np.dot(u_sat_pointing, u_vec_to_earth), -1.0, 1.0)
115     angle_earth = np.arccos(cos_angle_earth)
116     apparent_radius_earth = np.arctan(EARTH_RADIUS / dist_to_earth) if dist_to_earth > 0
117     else 0
118     if (angle_earth - apparent_radius_earth) < earth_excl_angle:
119         earth_flag = True
120
121     if print_debug:
122         print(f"--- Exclusion Debug for Satellite {satellite_index} ---")
123         print(f"    - Sun Flag: {sun_flag}, Angle: {np.rad2deg(angle_sun):.2f} deg, Excl
124         : {np.rad2deg(solar_excl_angle):.2f} deg")
125         print(f"    - Moon Flag: {moon_flag}, Angle: {np.rad2deg(angle_moon):.2f} deg,
126         Excl: {np.rad2deg(lunar_excl_angle):.2f} deg")
127         print(f"    - Earth Flag: {earth_flag}, Angle: {np.rad2deg(angle_earth):.2f} deg,
128         Excl: {np.rad2deg(earth_excl_angle):.2f} deg")
129
130     is_excluded = sun_flag or moon_flag or earth_flag
131     return 0 if is_excluded else 1
132
133 def update_visibility_table(
134     data_struct: Dict[str, Any],
135     print_debug_for_sat: Optional[int] = None
136 ) -> None:
137     """
138     Updates the visibility table for each satellite against each fixed point.
139
140     Args:
141         data_struct: The main simulation data dictionary.
142         print_debug_for_sat: If an integer is provided, the 'exclusion' function's
143             debug printout will be enabled for that satellite index.
144     """
145     num_satellites = data_struct['counts']['satellites']
146     fixed_points = data_struct['fixedpoints']['position']
147     num_fixed_points = len(fixed_points)
148     visibility_table = data_struct['fixedpoints']['visibility']
149
150     if num_satellites == 0 or num_fixed_points == 0:
151         return
152
153     satellite_positions = data_struct['satellites']['position']
154
155     for i in range(num_satellites):
156         sat_pos = satellite_positions[i]
157         should_print_debug = (i == print_debug_for_sat)
158         for j in range(num_fixed_points):
159             fixed_point_pos = fixed_points[j]
160             pointing_vector = fixed_point_pos - sat_pos

```

```
153         data_struct['satellites']['pointing'][i] = pointing_vector
154         visibility_table[j, i] = exclusion(data_struct, i, print_debug=
should_print_debug)
```